

The background features a stylized, glowing laptop in shades of blue and purple. The laptop is semi-transparent, revealing internal components like the keyboard and screen. It is surrounded by a pattern of glowing blue hexagons and dots, giving it a digital or technological feel. The overall color scheme is dark blue with bright blue and purple highlights.

Escuela : Tecnologías de la Información

Seminario

Complementario

Módulos y paquetes para Machine Learning con Python

Presentación

Nombre: Stefani Rojas Zuñiga

Especialidad: Transformación Digital, BPMN y Gestión de Proyectos TI.

Trayectoria: +10 años liderando infraestructura, automatización y mejora continua en los sectores financiero y educativo.



Participa!

Cuéntame **tu nombre** y si **ya estás haciendo prácticas** o en **qué área te gustaría trabajar**.

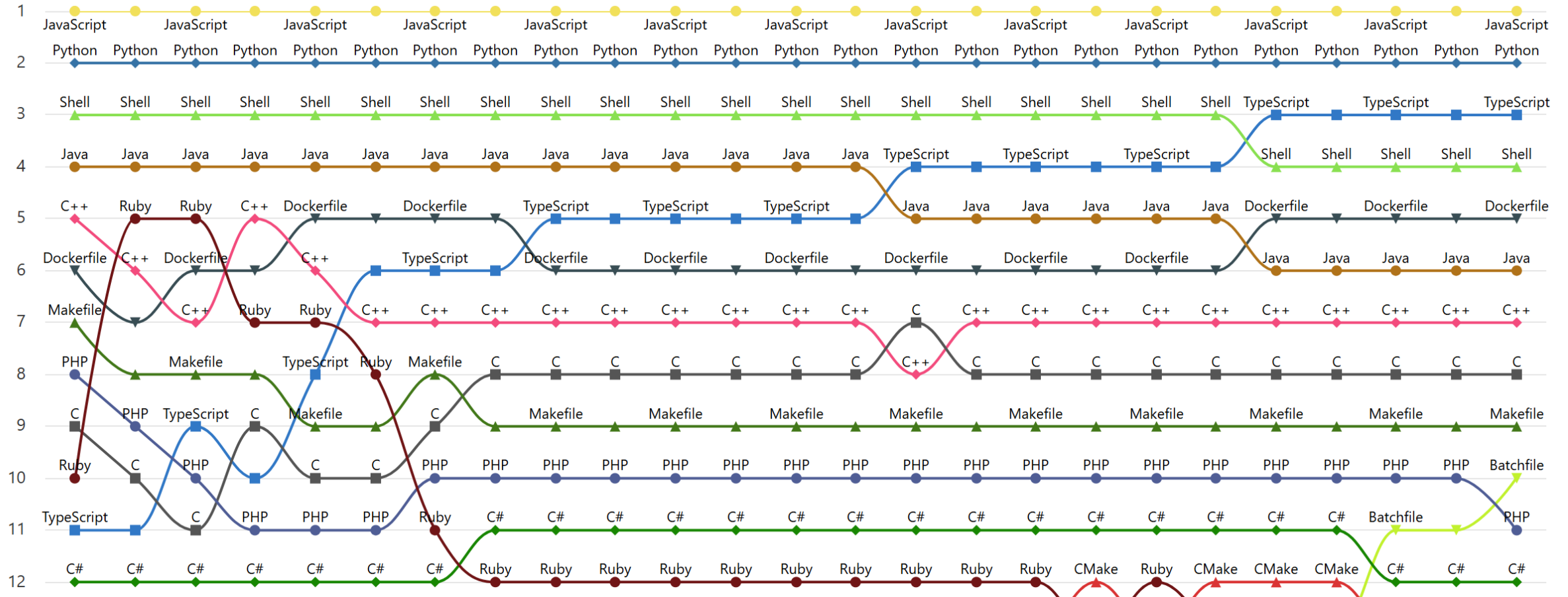
¿Qué es Python?

Python es un **lenguaje de programación** de alto nivel, interpretado y de propósito general.

¿Para qué sirve Python?

- Desarrollo web (crear páginas y aplicaciones web como Instagram o Spotify).
- Ciencia de datos e inteligencia artificial (es el rey en machine learning, análisis de datos y visualización).
- Automatización de tareas (crear scripts que hagan tu trabajo tedioso por ti, como renombrar miles de archivos).
- Desarrollo de videojuegos (aunque no es el más rápido, se usa para prototipos y lógica).
- Software educativo y científico (cálculo numérico, biología, física, etc.).

Python



Fuente: [Programming languages | GitHub Innovation Graph](#)

¿Qué es Machine Learning (ML)?

Machine Learning es la ciencia de hacer que las computadoras aprendan y tomen decisiones por sí mismas, sin ser programadas explícitamente para cada tarea.

Es decir, en lugar de escribir un programa con reglas fijas del tipo “*Si el cliente gana más de \$50.000 Y tiene menos de 30 años, ENTONCES ofrecerle crédito*”, tú le **muestras muchos ejemplos** al ordenador y él **solito encuentra los patrones** para tomar esa decisión.

Machine Learning con Python

1. RECOLECCIÓN DE DATOS



Archivos



Bases de datos



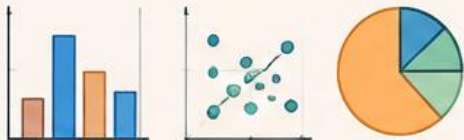
APIs / Web

2. PREPARACIÓN DE DATOS



- Limpieza
- Transformación
- Normalización
- Selección de características

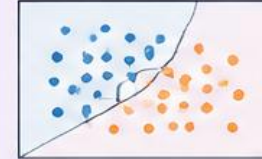
3. EXPLORACIÓN Y ANÁLISIS



- Estadísticas descriptivas
- Visualización de datos



4. ENTRENAMIENTO DEL MODELO



- Regresión
- Clasificación
- Clustering
- Redes Neuronales
- Entre otros...

5. EVALUACIÓN DEL MODELO

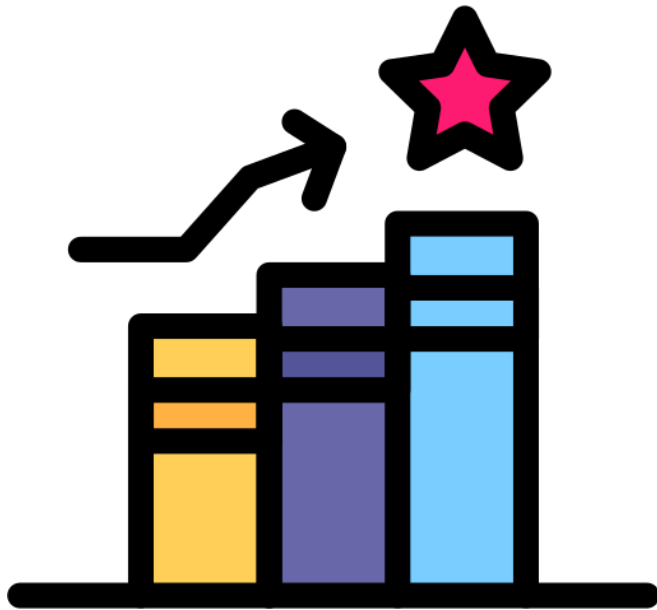
		Predicho	
Real		TP	FP
		FN	TN

- Precisión
- Exactitud
- Recall
- F1-Score
- Matriz de confusión

6. DESPLIEGUE Y PREDICCIÓN



- API / Aplicación
- Predicciones en tiempo real
- Toma de decisiones

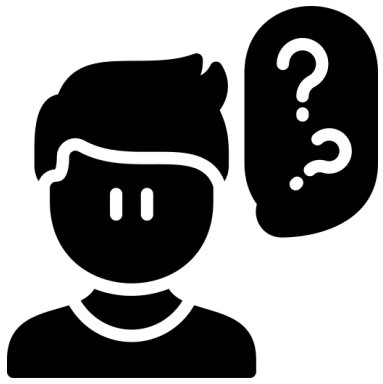


RESULTADO DE APRENDIZAJE:

- Implementa el entorno de desarrollo para Python y desarrolla programas básicos utilizando variables, operadores, estructuras de control y funciones, como base para el uso de librerías de Machine Learning.

CONTENIDO:

- Instalación de Anaconda y Visual Studio Code
- Uso de Jupyter Notebook
- Variables y tipos de datos
- Entrada y salida de datos
- Operadores
- Condicionales
- Bucles
- Funciones
- Importación de módulos



databricks



Anaconda es una distribución de Python orientada a Ciencia de Datos.

Es un entorno de Jupyter Notebook en la nube, totalmente gratuito, que corre en tu navegador.

Es un editor de código / IDE (Entorno de Desarrollo Integrado) creado por Microsoft.

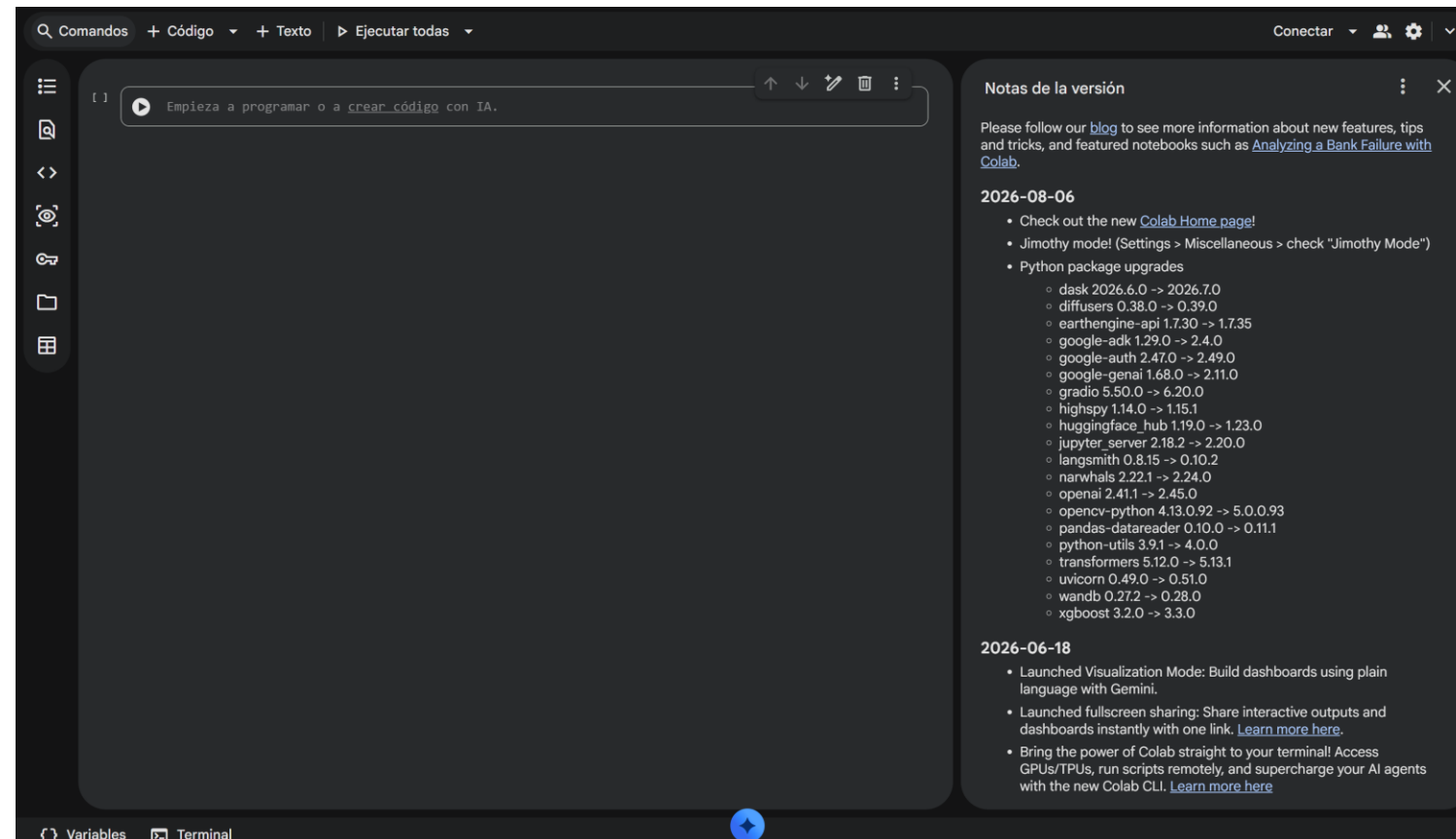
Entorno Cloud: Google Colab



Cero instalación, poder inmediato

Google Collaboratory ofrece un espacio de desarrollo en la nube basado en notebooks Jupyter, eliminando barreras de configuración inicial.

Perfecto para flujos rápidos, cuenta con acceso directo a hardware potente de forma gratuita (GPUs/TPUs), integración nativa con Google Drive y colaboración en tiempo real.

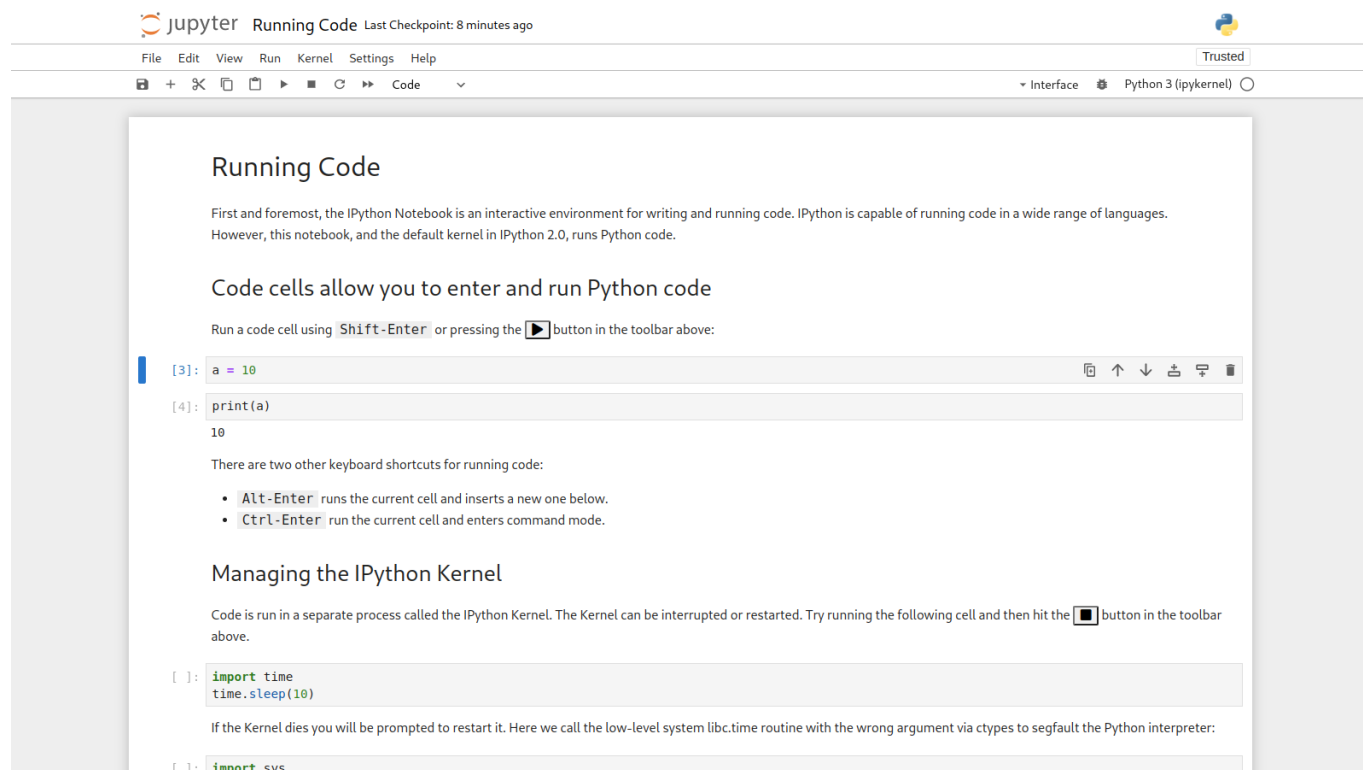


Cuaderno interactivo: Jupyter

Iteración rápida y documentada

Tanto en local (mediante Anaconda) como en la nube (Colab), el concepto de Notebooks de Jupyter redefine la forma de programar.

- Ejecuta bloques aislados de código, manteniendo el estado de las variables en memoria de forma persistente.
- Integra directamente código vivo con notas estructuradas en Markdown, ecuaciones complejas y gráficos visuales interactivos.



Variables y tipo de datos

¿Qué es una variable?

Es un espacio reservado en la memoria RAM del sistema. Al tener tipado dinámico, Python deduce automáticamente el tipo de la variable según su contenido.

¿Cuáles son los tipos de variables?

A

Numéricos: Enteros (int) y Decimales (float).

Textuales: Cadenas (str) delimitadas por comillas.

Lógicos: Booleanos (True o False).



Ejemplo de Sintaxis

Edad = 28

int

Tasa = 0.18

#float

Canal = "colab"

#str

Es_útil: True

#bool

Entrada y salida de datos

➡ Entrada: input()

La función input() pausa la ejecución del programa en la consola para recolectar lo que el usuario digite.

Siempre retorna texto (string).

Para procesar números, se requiere conversión explícita:

Captura de datos numéricos

```
edad= int (input("Ingresa tu edad"))
```

➡ Salida: print()

La función print() escribe texto o el contenido de variables directamente en pantalla de forma formateada.

Usa la sintaxis moderna de **f-strings** anteponiendo la letra f para incrustar variables:

Imprimir con formato directo

```
Print(f "sesión activa con: {edad} años")
```

Categoría	Operadores	Propósito	Ejemplo de Código
Aritméticos	+, -, *, /, **, //, %	Operaciones estándar, potencias, divisiones enteras y residuos.	residuo = 10 % 3 # 1
Comparación	==, !=, >, <, >=, <=	Evaluación lógica entre dos operandos retornando booleanos.	es_mayor = edad >= 18
Lógicos	and, or, not	Agrupación lógica de múltiples evaluaciones de verdad.	permitido = es_valido and activo

Control de Flujo: if, elif, else

Las condicionales permiten ejecutar bloques de instrucciones basándose únicamente en el cumplimiento de ciertas premisas lógicas de decisión.

🔑 **if:** Evaluación obligatoria de una condición.

🔑 **elif:** Evalúa nuevas opciones en cadena secuencial si las previas fallaron.

🔑 **else:** Código residual final si ninguna otra regla se cumplió.

🇲🇵 Regla de la Indentación

Python descarta el uso de llaves o palabras clave especiales para abrir bloques de instrucciones. Utiliza exclusivamente espacios en blanco al inicio de la línea (normalmente 4 espacios).

```
edad = 18
```

```
if edad >= 18:
```

```
    # Este bloque está indentado (4 espacios) porque depende del 'if'  
    print("Eres mayor de edad")  
    print("Puedes pasar")
```

```
print("Este mensaje siempre se ejecuta porque no está dentro del  
'if'")
```

Crear un programa que solicite una calificación y determine:

- Excelente
- Bueno
- Aprobado
- Desaprobado



Tiempo: 10 min.

Bucle: for

Excelente para iterar sobre un rango definido o estructuras indexadas (listas, tuplas, conjuntos de datos).

```
#Bucle iterador numérico
For i in range(1, 4):
    print(f "Iteracion: {i}")
```

Bucle: while

Ideal para bucles controlados de ejecución indefinida. Funciona de manera continua **mientras una condición sea True**.

```
#Bucle por condición interna
Contador = 1
While contador <=3:
    print(f "contando: {contador}")
    contador +=1
```



1. Definición

Declaradas usando la palabra reservada **def**. Sirven para aislar y encapsular fragmentos de código repetitivos, promoviendo el principio DRY (Don't Repeat Yourself).



2. Parámetros

Variables declaradas en la firma de la función. Le permiten recibir entradas dinámicas del flujo principal para alimentar su propia lógica local.



3. Retorno

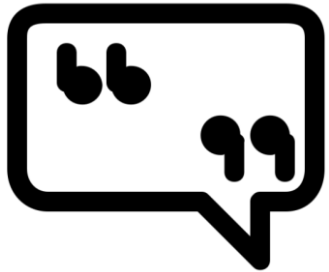
Mediante el uso de la cláusula **return**, la función detiene su propio subproceso y devuelve el valor procesado al punto inicial donde fue llamada.

```
#declaración y llamada función limpia
def calcular_neto (sueldo + bonos):
    return sueldo + bonos
Total = calcular_neto (2500, 300) #total almacena 2800
```

Reutilización de código existente

- ✓ **Sintaxis import:** Carga de librerías globales directamente a tu espacio de nombres local.
- ✓ **Librerías Estándar:** Bibliotecas integradas nativas como math (matemáticas avanzadas) o random (aleatoriedad).
- ✓ **Librerías de Terceros:** Instalables desde Anaconda o terminal para el control de datos masivos.

```
1 import tkinter as tk
2 from tkinter.constants import *
3
4 root = tk.Tk()
5 root.title("Simple Counter")
6 root.geometry("250x100")
7
8 # variable is stored in the root object
9 root.counter = 0
10
11 # function for counting the clicks from the UI button element
12 def clicked():
13     root.counter += 1
14     L["text"] = "Button clicked: " + str(root.counter)
15
16
17 b = tk.Button(root, text="Click Me", bootstyle=SUCCESS, command=clicked)
18 b.pack()
19
20 L = tk.Label(root, text="No clicks yet.")
21 L.pack()
22
23 root.mainloop()
```



“Los sistemas de información no solo mueven datos... mueven ideas, decisiones y el futuro de las organizaciones”